Break It, Find It, Fix It: Hunting Memory Leaks and Performance Bottlenecks

•••

Workshop

Jacek Psiuk

- Software Engineer with 15+ years of experience, voluum.com, Commerce Media Tech, Team Internet Group
- Building distributed systems in Java (and Kotlin, Python, C++)
- Computer Science Graduate, AGH University of Krakow
- Certified AWS Solutions Architect Professional



Download resources

- The MAT - https://eclipse.dev/mat/
- JMC - https://jdk.java.net/jmc/
- Java Object Layout (JOL) plugin for IntelliJ - https://github.com/stokito/IdeaJol

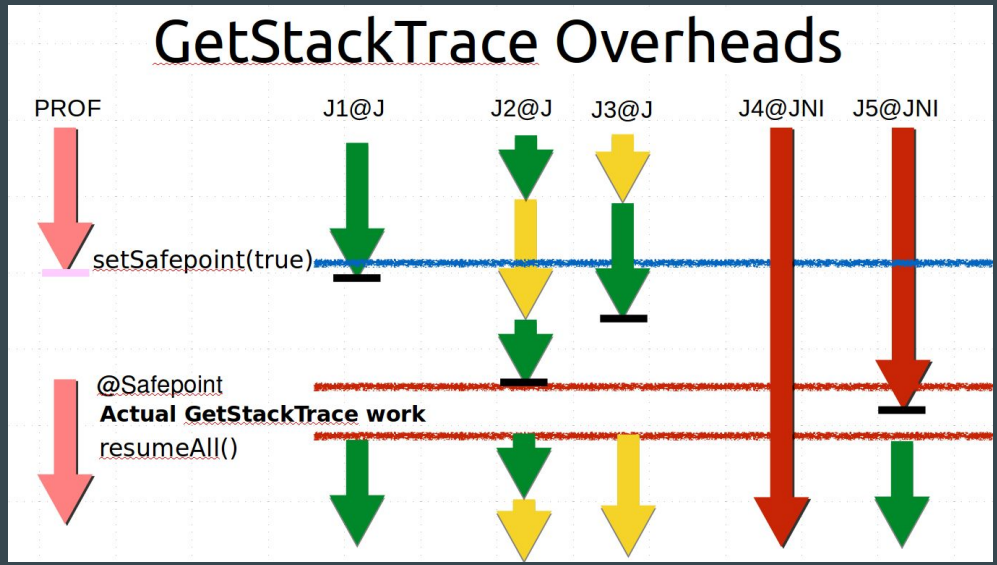
Tools for static analysis

Thread dump

- Copy of each JVM thread state
- It's good to start the analysis with
- To grasp thread's behavior in time, worth taking 2 with ~10-second gaps
- How
  - `$ jcmd PID Thread.print > thread.dump`
  - `$ jstack [options] pid` – This command is experimental and unsupported (docs)
  - OS-specific signals (linux: kill -3, ctrl+\)
- Impact: medium - depends on the number of threads (docs)
  - Threads must reach a "safepoint"

Thread dump - safepoint

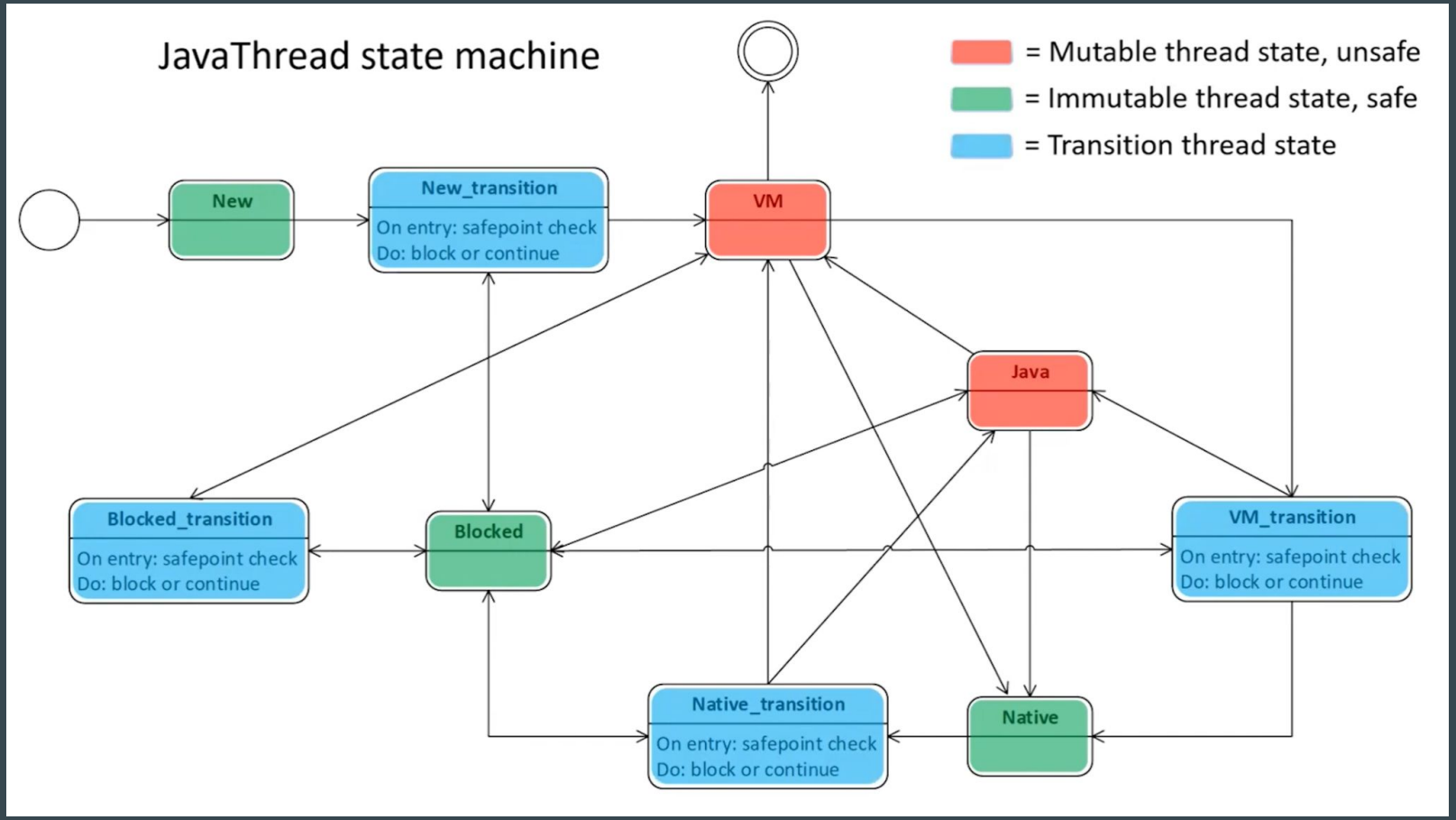
- A range of execution where the state of the executing thread is well described
  - Can't mutate the heap nor thread-local GC roots
- Used widely by the JVM
  - GC, interpreters, (de)compilation, JNI, debugging, profiling
- Time to get to a safepoint depends on the last thread to block
  - The "Safepoint bias" problem



Safepoints: Meaning, Side Effects and Overheads

Thread dump - safepoint

- Cooperative suspension
  - Publish a JVM-wide flag meaning a stop is requested
  - Each thread, between doing its work, checks for the value of the flag
    - Before calling methods
    - Before exiting from methods (`return` and `throw`)
    - Compiled code has special checks inserted
    - Loop back edges (check UseCountedLoopSafepoints or LoopStripMiningIter* JVM flags)
  - When a thread is blocked, it won't enter a mutable state
- Used widely by the JVM
  - GC, interpreters, (de)compilation, JNI, debugging, profiling



HotSpot JVM Deep Dive - Safepoint

Heap dump

- A snapshot of objects in the heap at a certain point in time
  - May or may not contain unreachable objects
  - All objects (class, fields primitive values and references)
  - All classes (name, super class, static fields, classloader)
  - GC roots
  - Thread stacks and local variables
- How
  - `jcmd <pid> GC.heap_dump <filename.hprof>`
  - `jmap -dump:format=b,file=<filename.hprof> <pid>`
  - `-XX:+HeapDumpOnOutOfMemoryError`
- Impact: high - depend on heap size; contains a full GC (unless -all) and STW
- Tools for heap analysis
  - Memory Analyzer Tool (MAT), Java VisualVM, IntelliJ Profiler

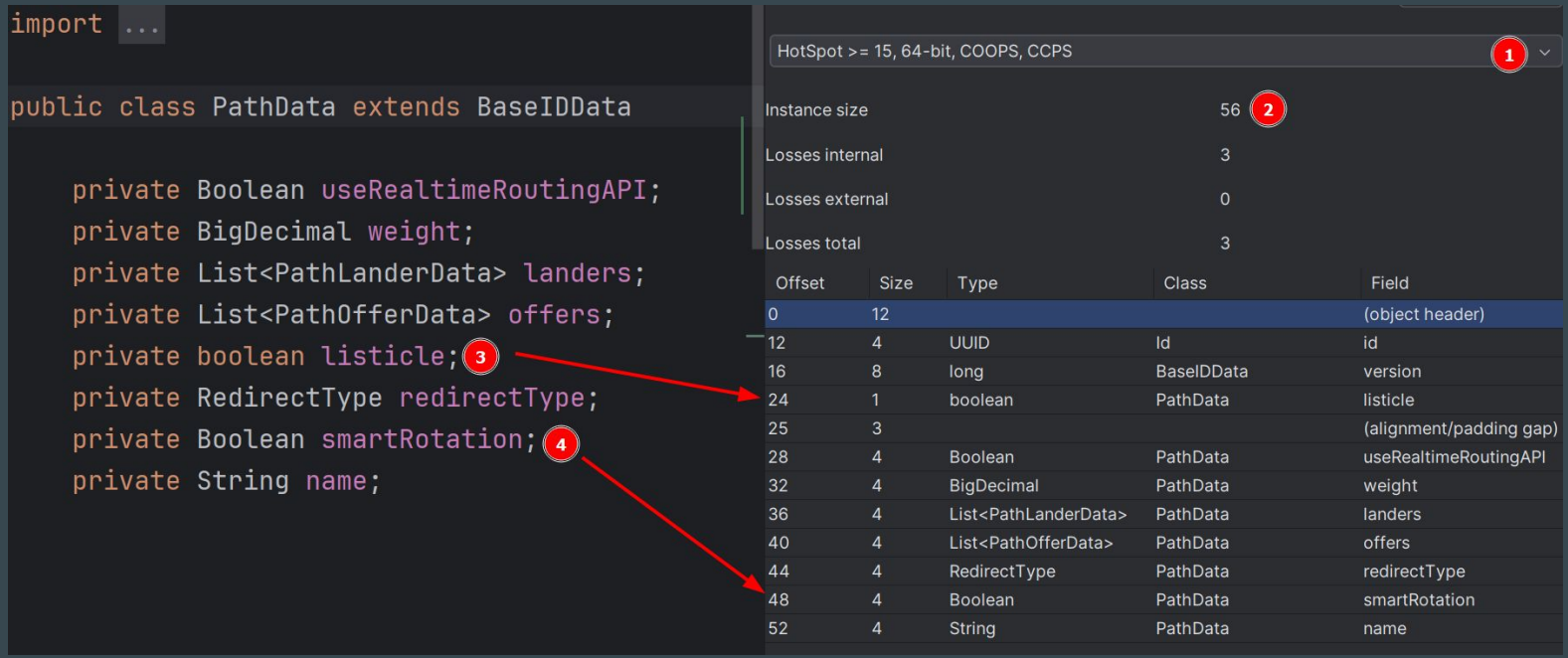
Heap dump - concepts

- Shallow heap - memory consumed by one object
  - Object header - 12-16 bytes (8 bytes with experimental `-XX:+UseCompactObjectHeaders`)
  - Primitive fields of according size (boolean/byte 1, int 4, long 8, double 8)
  - Reference fields of 4 or 8 bytes
  - Optional padding between fields (field of size N must be aligned by N)
  - Optional padding for alignment of the next object by 8 (or more for heaps of 32+ gb)

JVM Anatomy Quark #24: Object Alignment

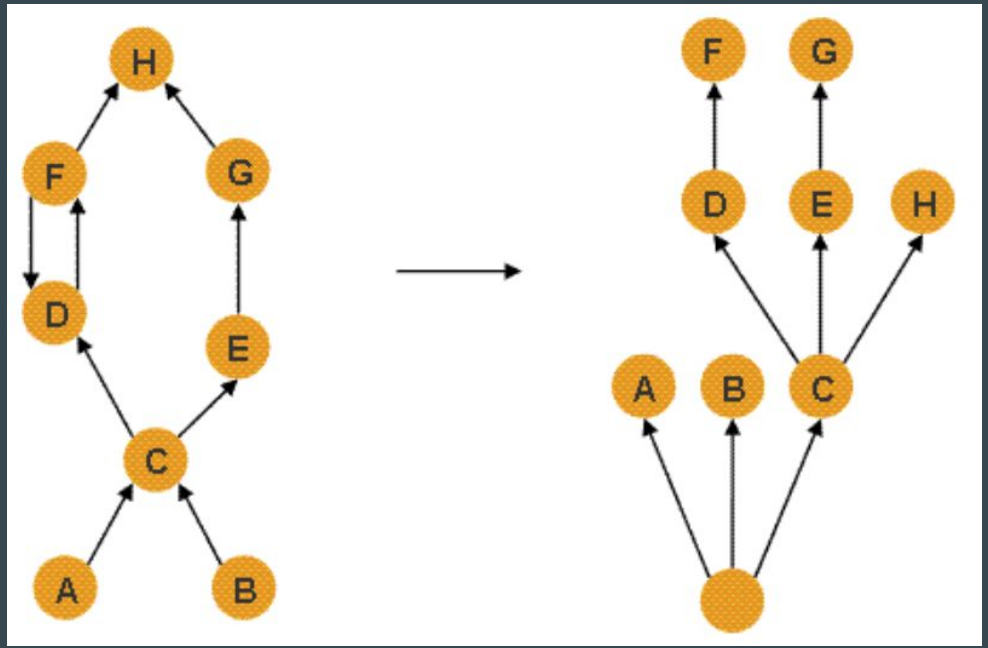
Heap dump - concepts

- Shallow heap - memory consumed by one object
  - Java Object Layout - IntelliJ plugin



Heap dump - concepts

- Shallow heap - memory consumed by one object
- Retained set of X - a set of objects which would be GCed if X was GCed
- Retained heap X - a sum of shallow heap of retained set
- Dominator tree
  - An object X dominates an object Y if every path from roots to Y goes through X
  - Edges do not correspond to objects references
  - Sub-tree of X is the retained set of X

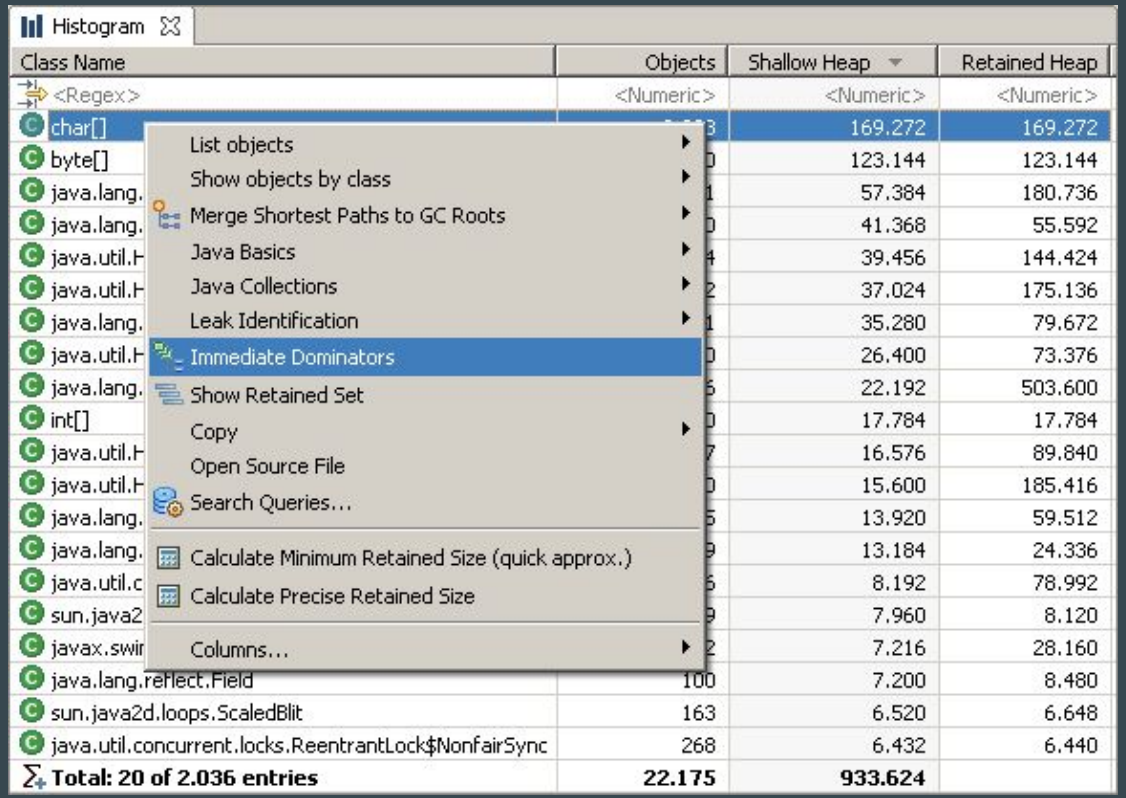


Heap dump - Memory Analyzer (MAT)

- Histogram - lists number of instances per class
  - Shallow heap - memory consumed by one object
  - Retained heap - sum of shallow heaps of objects that would be GC-ed if this object would be GC-ed
- Top consumers - print the most expensive objects
- Automatically identify leak suspects
- Thread analysis
  - Browse all the threads and their state
  - List all local variables for each stack frame
  - Calculate shallow and retained heap for each stack frame

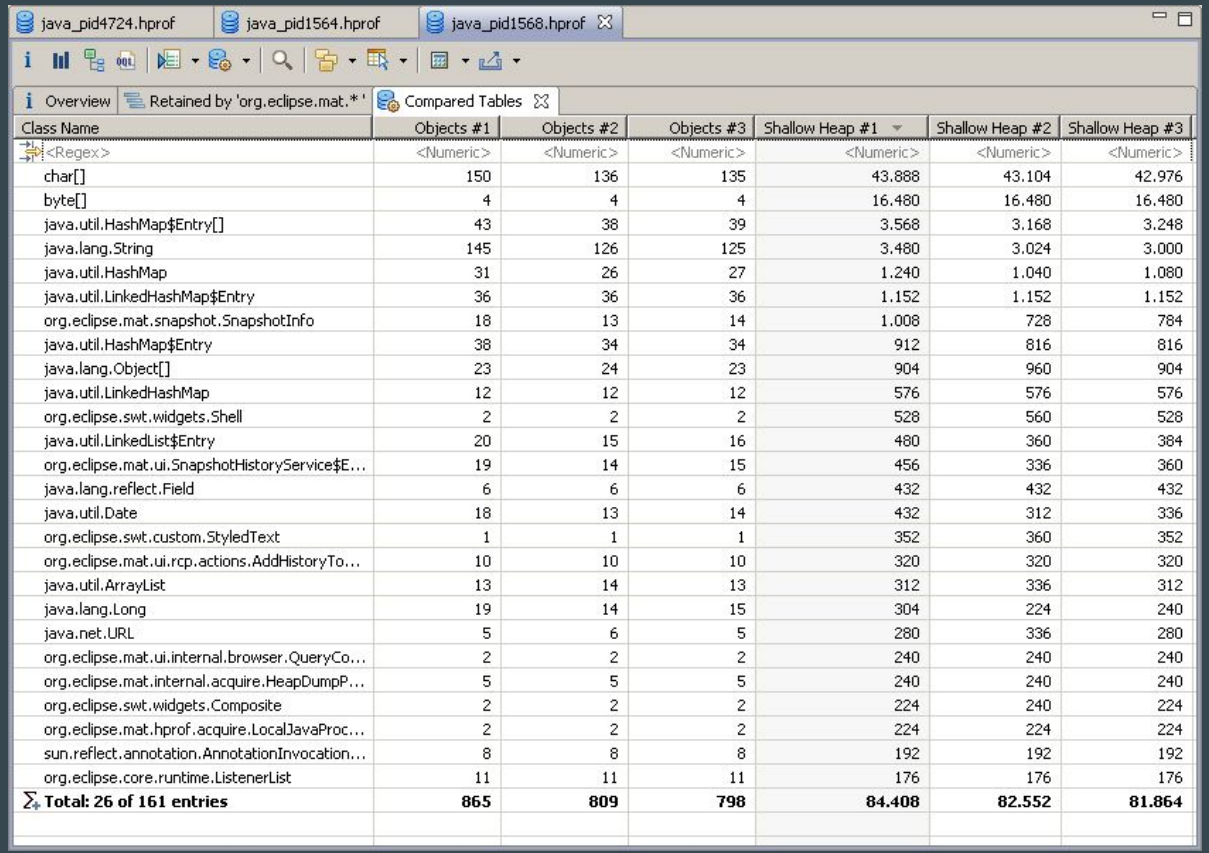
Heap dump - Memory Analyzer (MAT)

- Dominator tree
  - Find memory allocation responsible objects
  - List outgoing or incoming references of objects
  - Read values of all fields of objects
- Analyze Java collections usage
  - Distribution histogram of given arrays grouped by the size
  - Frequency distribution of fill ratios of collections
  - Frequency distribution of the collision ratios of Maps



Heap dump - Memory Analyzer (MAT) - Comparing heap dumps

- Take 2 or 3 heap dumps, load them into MAT and compare
- Compare aggregated results only on the class level
  - Objects IDs cannot be compared, they are addresses and objects can be moved during GC



Class Name	Objects #1	Objects #2	Objects #3	Shallow Heap #1	Shallow Heap #2	Shallow Heap #3
<Regex>	<Numeric>	<Numeric>	<Numeric>	<Numeric>	<Numeric>	<Numeric>
char[]	150	136	135	43.888	43.104	42.976
byte[]	4	4	4	16.480	16.480	16.480
java.util.HashMap$Entry[]	43	38	39	3.568	3.168	3.248
java.lang.String	145	126	125	3.480	3.024	3.000
java.util.HashMap	31	26	27	1.240	1.040	1.080
java.util.LinkedHashMap$Entry	36	36	36	1.152	1.152	1.152
org.eclipse.mat.snapshot.SnapshotInfo	18	13	14	1.008	728	784
java.util.HashMap$Entry	38	34	34	912	816	816
java.lang.Object[]	23	24	23	904	960	904
java.util.LinkedHashMap	12	12	12	576	576	576
org.eclipse.swt.widgets.Shell	2	2	2	528	560	528
java.util.LinkedList$Entry	20	15	16	480	360	384
org.eclipse.mat.ui.SnapshotHistoryService$E...	19	14	15	456	336	360
java.lang.reflect.Field	6	6	6	432	432	432
java.util.Date	18	13	14	432	312	336
org.eclipse.swt.custom.StyledText	1	1	1	352	360	352
org.eclipse.mat.ui.rcp.actions.AddHistoryTo...	10	10	10	320	320	320
java.util.ArrayList	13	14	13	312	336	312
java.lang.Long	19	14	15	304	224	240
java.net.URL	5	6	5	280	336	280
org.eclipse.mat.ui.internal.browser.QueryCo...	2	2	2	240	240	240
org.eclipse.mat.internal.acquire.HeapDumpP...	5	5	5	240	240	240
org.eclipse.swt.widgets.Composite	2	2	2	224	240	224
org.eclipse.mat.hprof.acquire.LocalJavaProc...	2	2	2	224	224	224
sun.reflect.annotation.AnnotationInvocation...	8	8	8	192	192	192
org.eclipse.core.runtime.ListenerList	11	11	11	176	176	176
Total: 26 of 161 entries	**865**	**809**	**798**	**84.408**	**82.552**	**81.864**

Heap dump - MAT - Object Query Language (OQL)

- Query the heap dump with custom SQL-like queries
- OQL represents classes as tables, objects as rows, and fields as columns
- Examples
  - select all int arrays of length 256 or more
    - `select a from [I a where a.length >= 256`
  - show content of Strings that match a regular expression
    - `select s.value.toString() from java.lang.String s where /java/.test(s.value.toString())`
  - show path value of all File objects
    - `select file.path.value.toString() from java.io.File file`

Native memory tracking - NMT

- Tracks memory used by JVM internals
  - Adds 2 machine words to all malloc memory as a malloc header
  - A 5-10% JVM performance drop
- Categories
  - Heap, Class, Thread, Code, GC, Compiler, Metaspace, NMT, Other
- How
  - `-XX:NativeMemoryTracking=detail`
  - `jcmd PID VM.native_memory detail scale=MB`
- To compare over time
  - `jcmd PID VM.native_memory baseline`
  - `jcmd PID VM.native_memory detail.diff`

Native memory tracking - NMT

- Reserved memory
  - A block of virtual address space that is set aside for future use
- Committed memory
  - A part of the reserved memory that has been assigned a backing store, either in physical RAM or the page file

```
Native Memory Tracking:

(Omitting categories weighting less than 1MB)

Total: reserved=11594MB, committed=11192MB (1)
       malloc: 180MB #1315366 (2)
       mmap:   reserved=11414MB, committed=11012MB (3)

-              (4) Java Heap (reserved=10240MB, committed=10240MB)
                             (mmap: reserved=10240MB, committed=10240MB)

-                (5) Class (reserved=212MB, committed=25MB)
                           (classes #33741)
                           (  instance classes #31801, array classes #1940)
                           (malloc=4MB #107138) (peak=4MB #104026)
                           (mmap: reserved=208MB, committed=21MB)
                           (  Metadata:   )
                           (    reserved=192MB, committed=154MB)
                           (    used=153MB)
                           (    waste=0MB =0.31%)
                           (  Class space:)
                           (    reserved=208MB, committed=21MB)
                           (    used=20MB)
                           (    waste=1MB =2.90%)

-                (6) Thread (reserved=105MB, committed=39MB)
                           (thread #332)
                           (stack: reserved=104MB, committed=38MB)
                           (malloc=1MB #1995) (peak=1MB #2376)
```

- $ pmap PID
- $ jcmd PID VM.native_memory detail

```
0000000580000000 10476544K rw---   [ anon ]
00000007ff700000    468K rw--- classes.jsa
00000007ff775000    556K rw---   [ anon ]
00000007ff800000    476K rw--- classes.jsa
00000007ff877000   7716K rw---   [ anon ]
000000b800000000  12160K rw--- classes.jsa
000000b800be0000   4224K -----   [ anon ]
000000b801000000    256K rw---   [ anon ]
000000b801040000    768K rw---   [ anon ]
000000b801100000   1280K rw---   [ anon ]
000000b801240000   1024K rw---   [ anon ]
000000b801340000    256K rw---   [ anon ]
000000b801380000   1792K rw---   [ anon ]
000000b801540000    256K rw---   [ anon ]
000000b801580000    896K rw---   [ anon ]
000000b801660000    128K rw---   [ anon ]
000000b801680000    256K rw---   [ anon ]
000000b8016c0000    256K rw---   [ anon ]
:

0000ffffb8882000    688K r-x-- libm-2.26.so
0000ffffb892e000     76K ----- libm-2.26.so
0000ffffb8941000      4K r---- libm-2.26.so
0000ffffb8942000      4K rw--- libm-2.26.so
0000ffffb8943000     28K r-x-- librt-2.26.so
0000ffffb894a000     96K ----- librt-2.26.so
0000ffffb8962000      4K r---- librt-2.26.so
0000ffffb8963000      4K rw--- librt-2.26.so
0000ffffb8964000  17800K r-x-- libjvm.so
0000ffffb9ac6000    104K ----- libjvm.so
0000ffffb9ae0000    848K r---- libjvm.so
0000ffffb9bb4000    196K rw--- libjvm.so
0000ffffb9be5000    504K rw---   [ anon ]
0000ffffb9c63000   1456K r-x-- libc-2.26.so
0000ffffb9dcf000     64K ----- libc-2.26.so
0000ffffb9ddf000     16K r---- libc-2.26.so
0000ffffb9de3000      8K rw--- libc-2.26.so
0000ffffb9de5000     16K rw---   [ anon ]
:
```

```
Virtual memory map:

[0x0000000580000000 - 0x0000000800000000] reserved and committed 10485760KB for
    [0x0000ffffb9773d0c] ReservedHeapSpace::try_reserve_range(char*, char*, unsi
    [0x0000ffffb9774750] ReservedHeapSpace::initialize_compressed_heap(unsigned
    [0x0000ffffb9774938] ReservedHeapSpace::ReservedHeapSpace(unsigned long, uns
    [0x0000ffffb972d8fc] Universe::reserve_heap(unsigned long, unsigned long)+0x

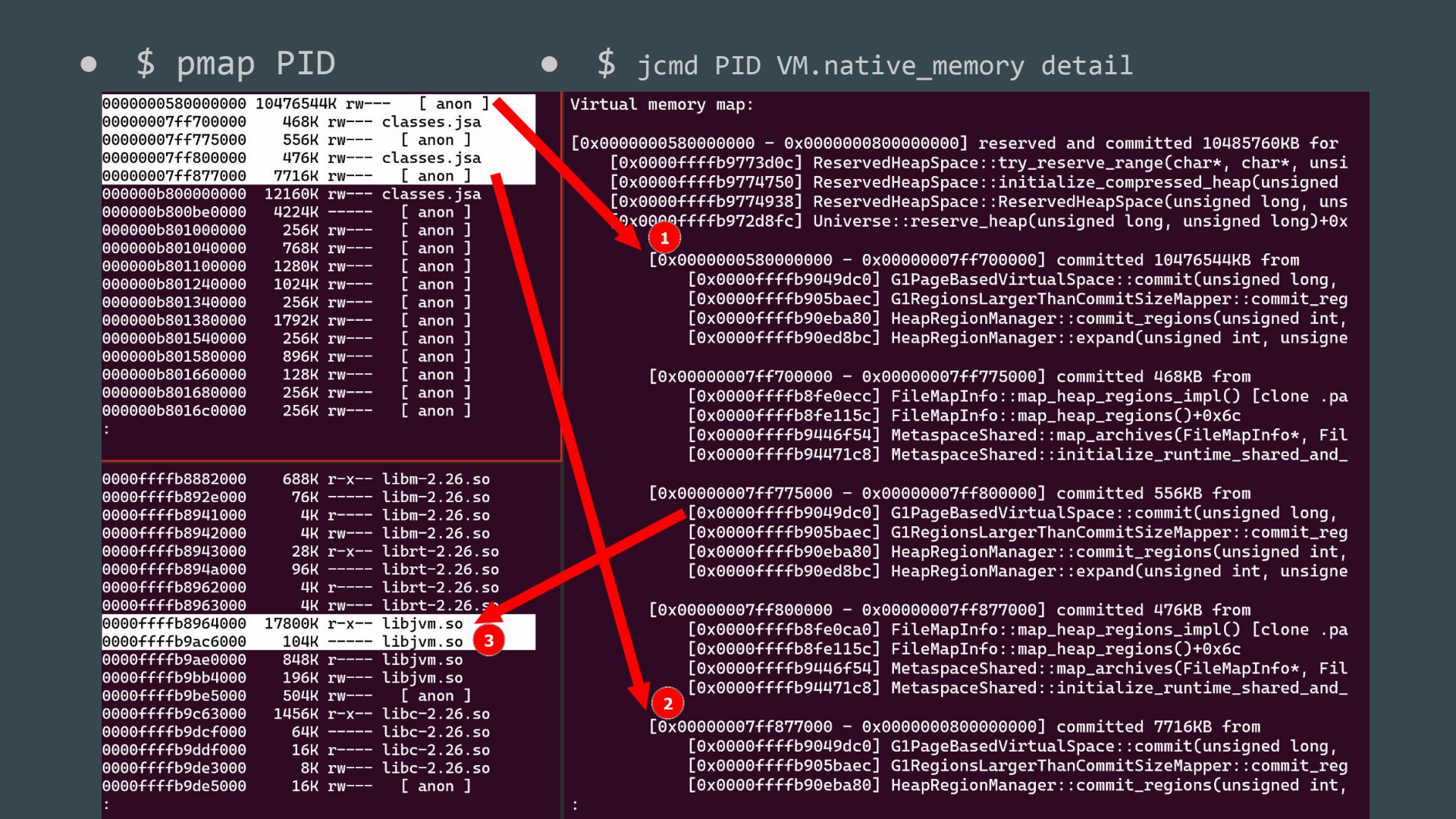
        [0x0000000580000000 - 0x00000007ff700000] committed 10476544KB from
            [0x0000ffffb9049dc0] G1PageBasedVirtualSpace::commit(unsigned long,
            [0x0000ffffb905baec] G1RegionsLargerThanCommitSizeMapper::commit_reg
            [0x0000ffffb90eba80] HeapRegionManager::commit_regions(unsigned int,
            [0x0000ffffb90ed8bc] HeapRegionManager::expand(unsigned int, unsigne

        [0x00000007ff700000 - 0x00000007ff775000] committed 468KB from
            [0x0000ffffb8fe0ecc] FileMapInfo::map_heap_regions_impl() [clone .pa
            [0x0000ffffb8fe115c] FileMapInfo::map_heap_regions()+0x6c
            [0x0000ffffb9446f54] MetaspaceShared::map_archives(FileMapInfo*, Fil
            [0x0000ffffb94471c8] MetaspaceShared::initialize_runtime_shared_and_

        [0x00000007ff775000 - 0x00000007ff800000] committed 556KB from
            [0x0000ffffb9049dc0] G1PageBasedVirtualSpace::commit(unsigned long,
            [0x0000ffffb905baec] G1RegionsLargerThanCommitSizeMapper::commit_reg
            [0x0000ffffb90eba80] HeapRegionManager::commit_regions(unsigned int,
            [0x0000ffffb90ed8bc] HeapRegionManager::expand(unsigned int, unsigne

        [0x00000007ff800000 - 0x00000007ff877000] committed 476KB from
            [0x0000ffffb8fe0ca0] FileMapInfo::map_heap_regions_impl() [clone .pa
            [0x0000ffffb8fe115c] FileMapInfo::map_heap_regions()+0x6c
            [0x0000ffffb9446f54] MetaspaceShared::map_archives(FileMapInfo*, Fil
            [0x0000ffffb94471c8] MetaspaceShared::initialize_runtime_shared_and_

        [0x00000007ff877000 - 0x0000000800000000] committed 7716KB from
            [0x0000ffffb9049dc0] G1PageBasedVirtualSpace::commit(unsigned long,
            [0x0000ffffb905baec] G1RegionsLargerThanCommitSizeMapper::commit_reg
            [0x0000ffffb90eba80] HeapRegionManager::commit_regions(unsigned int,
:
```



Profilers

JDK Flight Recorder - JFR

- An observability and monitoring framework built into the OpenJDK
- How
  - `jcmd <PID> JFR.start duration=10m name=profiling filename=recording.jfr settings=profile.jfc`
  - `-XX:StartFlightRecording:settings=default.jfc`
- View recording results
  - `$ jfr view [view-name or all-views] file`
  - `$ jfr view --help`
- Configuration files specify which events should be recorded
  - Two preset configurations: `default.jfc`, `profile.jfc`
  - You can create your own
- Impact: low (~1% default, <2% with profiling)
  - OK for prod, can be ongoing

JDK Flight Recorder - JFR

- `$ jfr view [view-name or all-views] file`
- `$ jfr view --help`

```
Java virtual machine views:

 blocked-by-system-gc      gc-concurrent-phases jvm-information

 class-modifications       gc-configuration     longest-compilations

 compiler-configuration    gc-cpu-time          native-memory-committed

 compiler-phases           gc-parallel-phases   native-memory-reserved

 compiler-statistics       gc-pause-phases      safepoints

 deoptimizations-by-reason gc-pauses            tlabs

 deoptimizations-by-site   gc-references        vm-operations

 gc                        heap-configuration
```

JDK Flight Recorder - JFR

- `$ jfr view [view-name or all-views] file`
- `$ jfr view --help`

```
Environment views:

 active-recordings        cpu-load              native-library-failures

 active-settings          cpu-load-samples      network-utilization

 container-configuration  cpu-tsc              recording

 container-cpu-throttling environment-variables system-information

 container-cpu-usage      events-by-count       system-processes

 container-io-usage       events-by-name        system-properties

 container-memory-usage   jvm-flags
```

JDK Flight Recorder - JFR

```
Application views:
 allocation-by-class            exception-by-type     monitor-inflation
 allocation-by-site             exception-count       native-methods
 allocation-by-thread           file-reads-by-path    object-statistics
 class-loaders                  file-writes-by-path   pinned-threads
 contention-by-address          finalizers            socket-reads-by-host
 contention-by-class            hot-methods           socket-writes-by-host
 contention-by-site             latencies-by-type     thread-allocation
 contention-by-thread           longest-class-loading thread-count
 deprecated-methods-for-removal memory-leaks-by-class thread-cpu-load
 exception-by-message           memory-leaks-by-site  thread-start
 exception-by-site              modules
```

JDK Flight Recorder - JFR

Java Methods that Executes the Most

Method	Samples	Percent
java.util.HashMap.getNode(Object)	437	5.80%
org.apache.tomcat.util.buf.MessageBytes.equalsIgnoreCase(String)	437	5.80%
java.lang.String.equalsIgnoreCase(String)	327	4.34%
sun.nio.cs.UTF_8$Encoder.encodeLoop(CharBuffer, ByteBuffer)	298	3.95%
java.util.concurrent.ConcurrentLinkedQueue.offer(Object)	296	3.93%
java.util.concurrent.ConcurrentLinkedQueue.size()	275	3.65%

JDK Flight Recorder - JFR

Thread Allocation Statistics

Thread	Allocated	Percentage
packet-queue-processor-2	192.1 GB	1.47%
packet-queue-processor-1	191.7 GB	1.47%
packet-queue-processor-0	191.6 GB	1.47%
http-nio-8080-exec-14	60.9 GB	0.47%
http-nio-8080-exec-49	60.9 GB	0.47%
http-nio-8080-exec-54	60.9 GB	0.47%

JDK Flight Recorder - JFR

```
GC Pauses

---------

Total Pause Time: 224 ms

Number of Pauses: 9

Minimum Pause Time: 0.0735 ms

Median Pause Time: 39.0 ms

Average Pause Time: 24.9 ms

P90 Pause Time: 48.6 ms

P95 Pause Time: 48.6 ms

P99 Pause Time: 48.6 ms

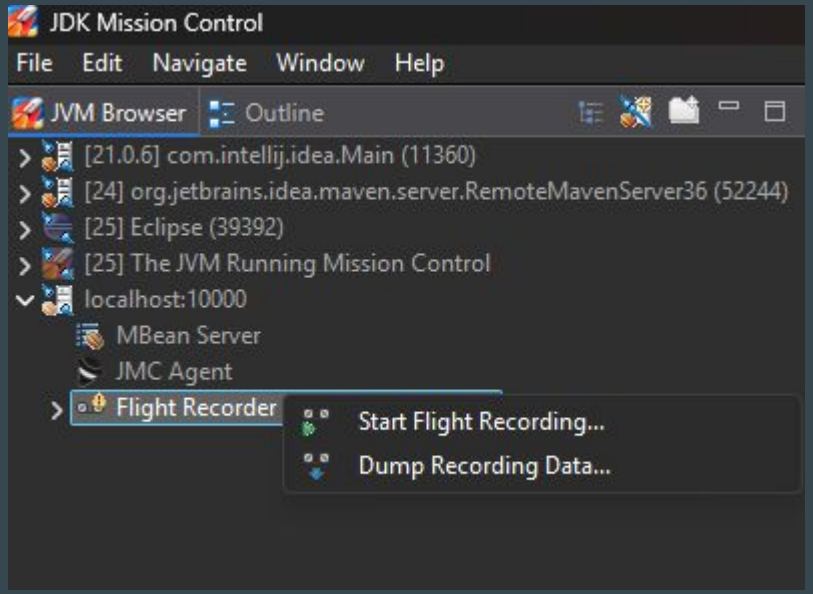
P99.9% Pause Time: 48.6 ms

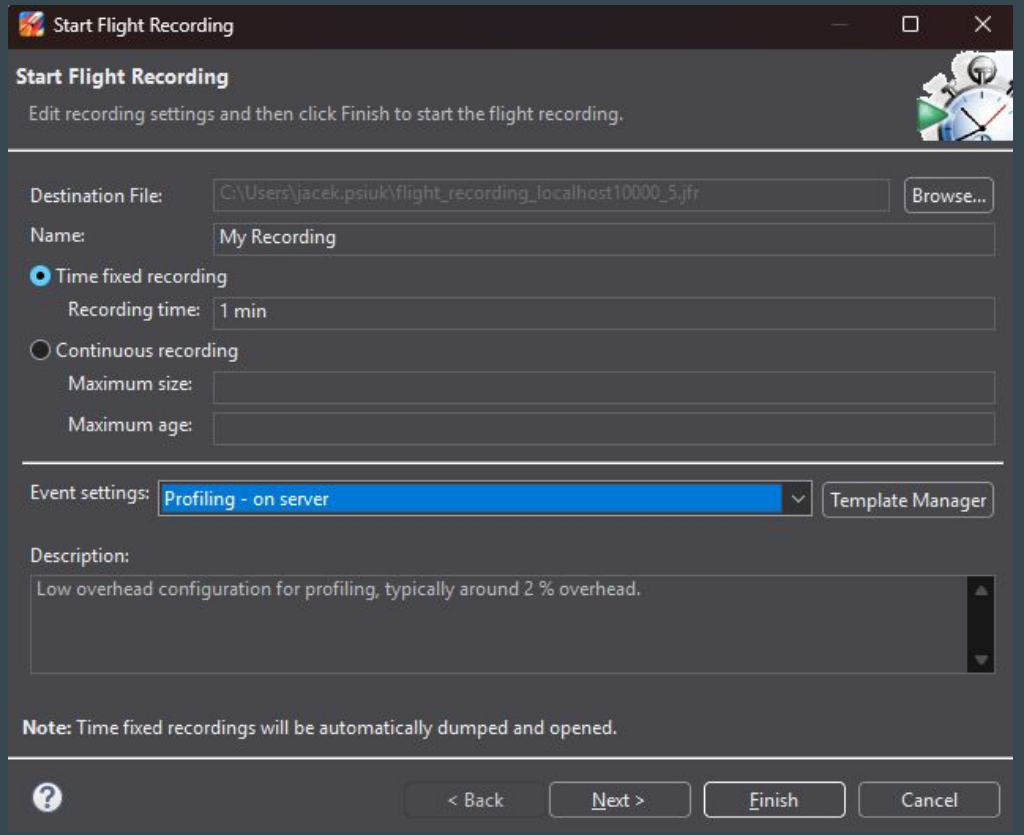
Maximum Pause Time: 48.6 ms
```

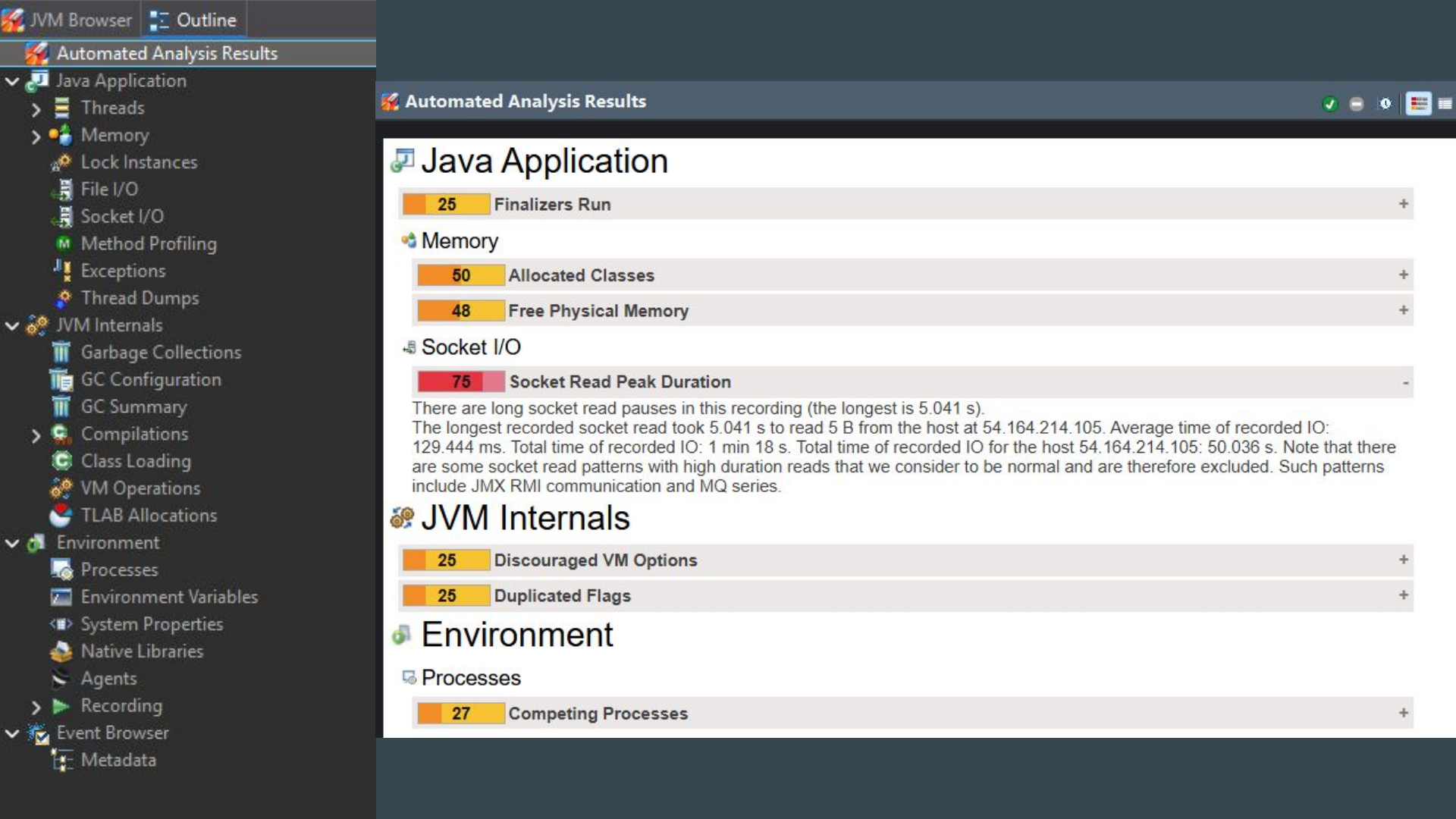
JDK Mission Control - JMC

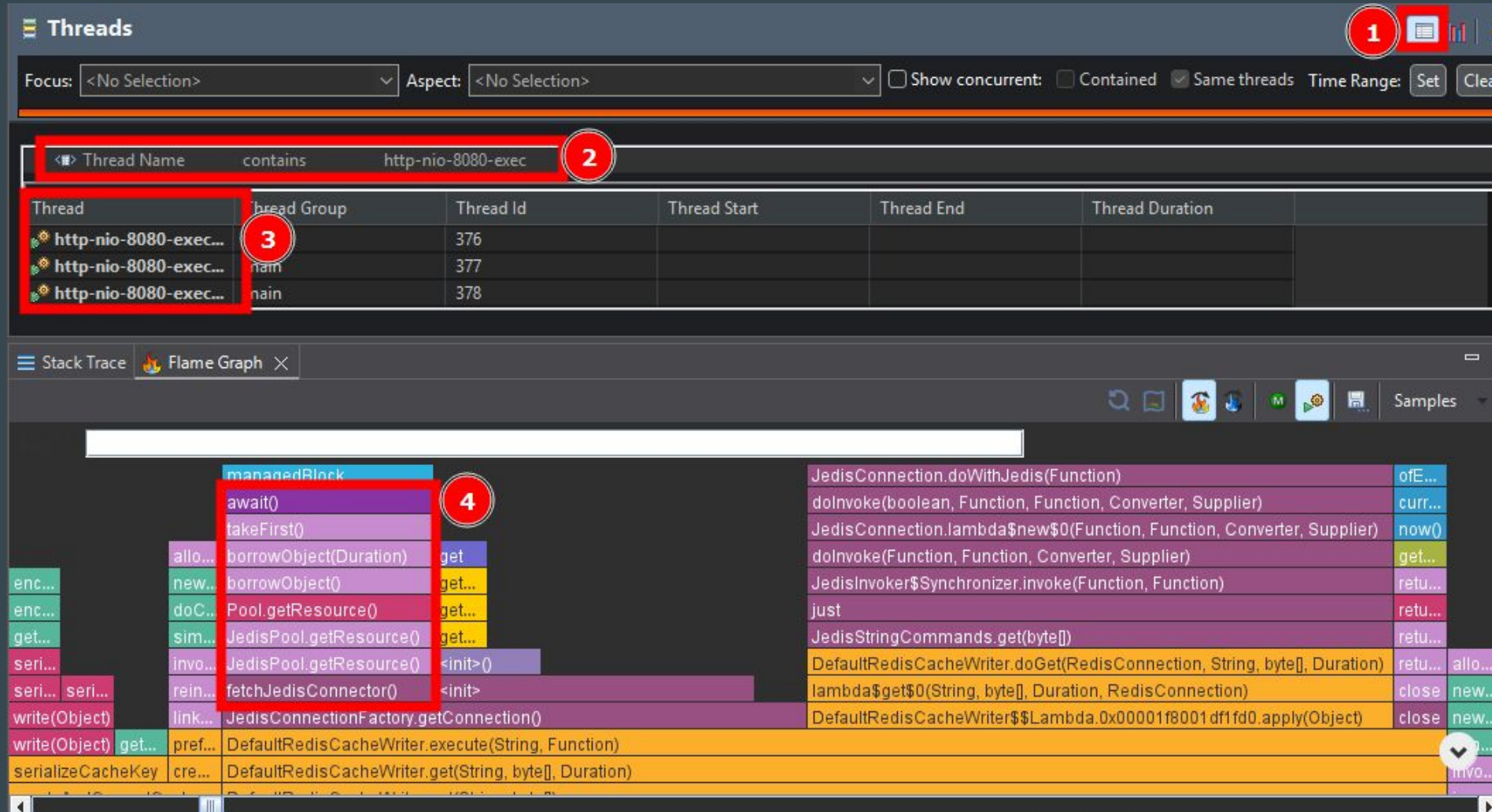
- A powerful tool for profiling and debugging applications running on a JVM
- Analyzes and visualizes JFR recordings
- Connects to JVM via JMX
  - `-Dcom.sun.management.jmxremote.port=9999`
    - Ensure no access to this port from the outside!
  - `-Dcom.sun.management.jmxremote.authenticate=false`
  - `-Dcom.sun.management.jmxremote.ssl=false`
  - `$ ssh <IP> -L 10000:127.0.0.1:9999`
  - Connect to: `service:jmx:rmi:///jndi/rmi://localhost:10000/jmxrmi`
- Flight Recorder → Start Flight Recording → Wait until it's done...

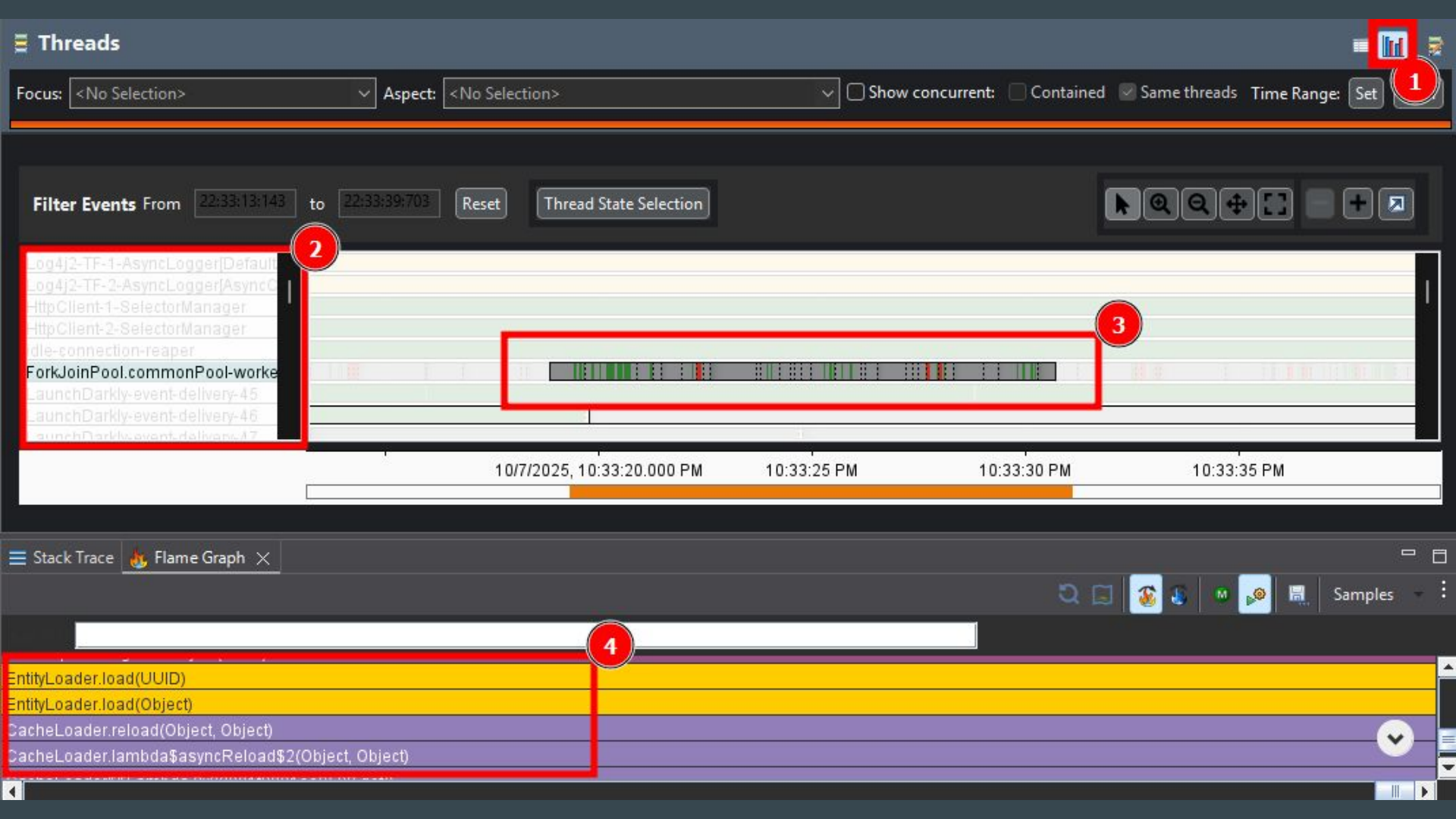
JDK Mission Control - JMC

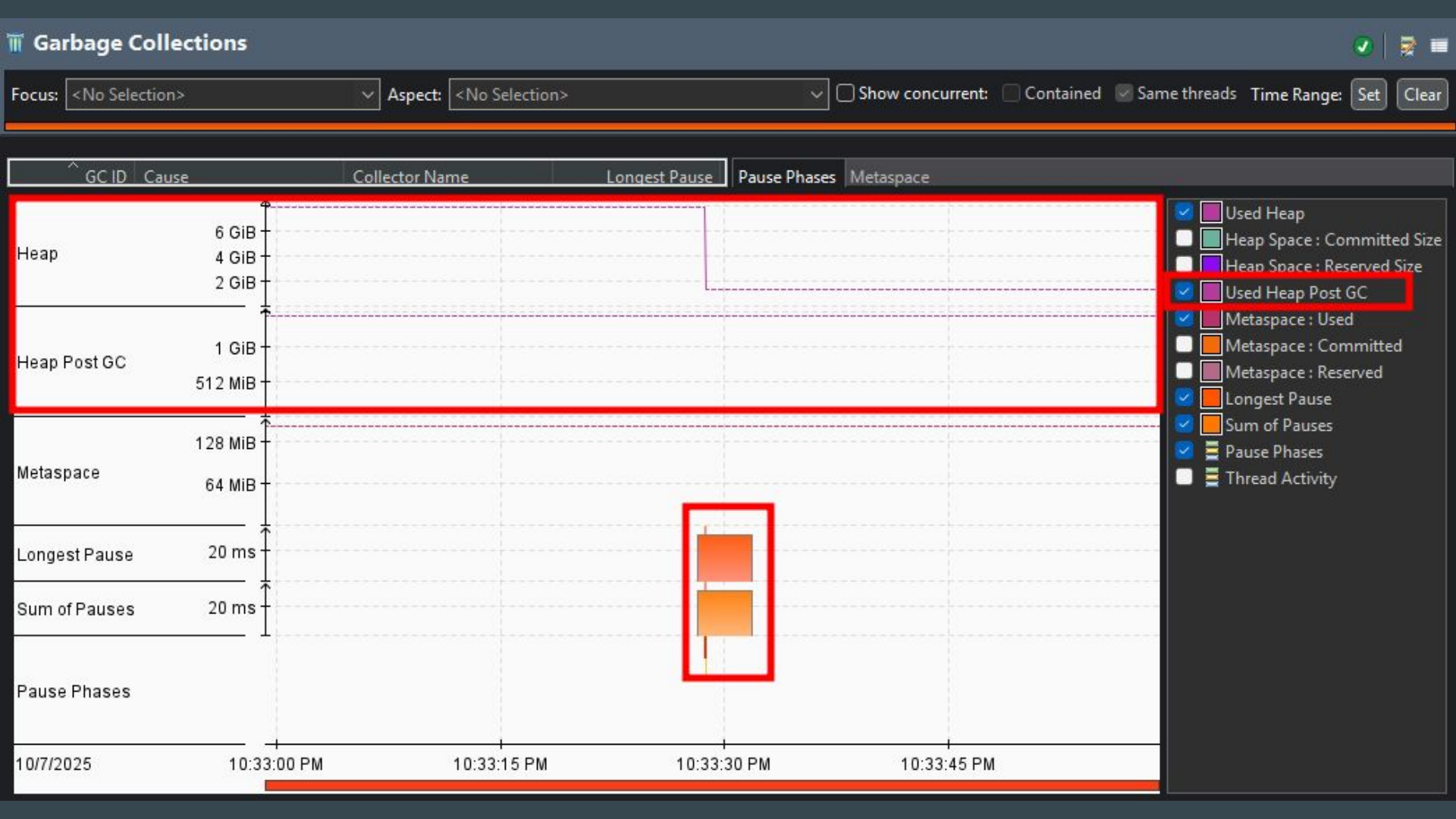


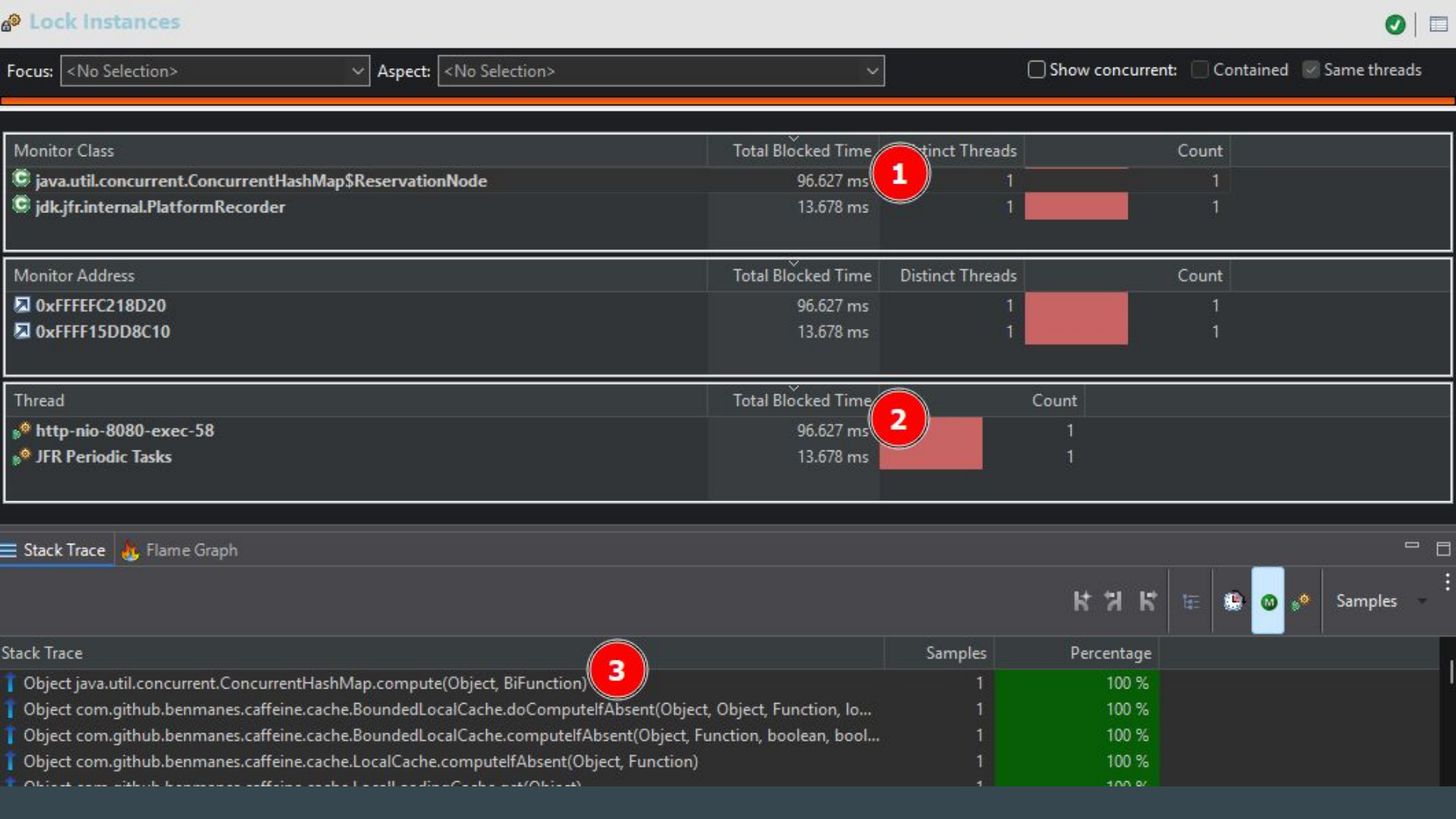


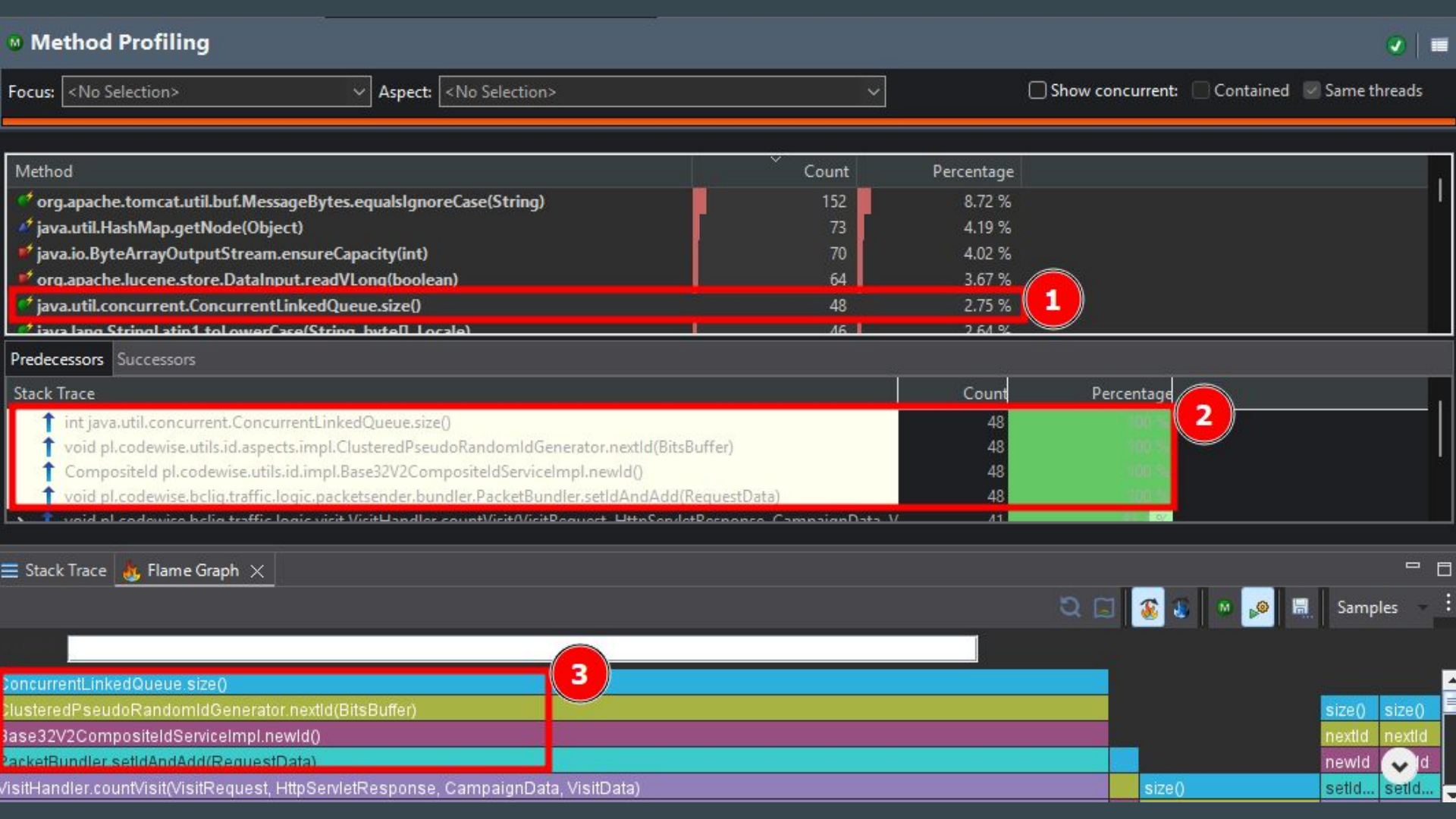

JVM Browser
Outline
Automated Analysis Results
Java Application
Threads
Memory
Lock Instances
File I/O
Socket I/O
Method Profiling
Exceptions
Thread Dumps
JVM Internals
Garbage Collections
GC Configuration
GC Summary
Compilations
Class Loading
VM Operations
TLAB Allocations
Environment
Processes
Environment Variables
System Properties
Native Libraries
Agents
Recording
Event Browser
Metadata
Automated Analysis Results
Java Application
25 Finalizers Run
Memory
50 Allocated Classes
48 Free Physical Memory
Socket I/O
75 Socket Read Peak Duration
There are long socket read pauses in this recording (the longest is 5.041 s).
The longest recorded socket read took 5.041 s to read 5 B from the host at 54.164.214.105. Average time of recorded IO: 129.444 ms. Total time of recorded IO: 1 min 18 s. Total time of recorded IO for the host 54.164.214.105: 50.036 s. Note that there are some socket read patterns with high duration reads that we consider to be normal and are therefore excluded. Such patterns include JMX RMI communication and MQ series.
JVM Internals
25 Discouraged VM Options
25 Duplicated Flags
Environment
Processes
27 Competing Processes


Threads
1
Focus: <No Selection>
Aspect: <No Selection>
Show concurrent:
Contained
Same threads
Time Range:
Set
Thread Name contains http-nio-8080-exec
2
Thread
Thread Group
Thread Id
Thread Start
Thread End
Thread Duration
http-nio-8080-exec...
3
376
http-nio-8080-exec...
377
http-nio-8080-exec...
378
Stack Trace
Flame Graph
Samples
managedBlock
await()
takeFirst()
borrowObject(Duration)
borrowObject()
Pool.getResource()
JedisPool.getResource()
JedisPool.getResource()
fetchJedisConnector()
JedisConnectionFactory.getConnection()
4
DefaultRedisCacheWriter.execute(String, Function)
DefaultRedisCacheWriter.get(String, byte[], Duration)
JedisConnection.doWithJedis(Function)
doInvoke(boolean, Function, Function, Converter, Supplier)
JedisConnection.lambdanew0(Function, Function, Converter, Supplier)
doInvoke(Function, Function, Converter, Supplier)
JedisInvoker$Synchronizer.invoke(Function, Function)
just
JedisStringCommands.get(byte[])
DefaultRedisCacheWriter.doGet(RedisConnection, String, byte[], Duration)
lambdaget0(String, byte[], Duration, RedisConnection)
DefaultRedisCacheWriter$$Lambda.0x00001f8001df1fd0.apply(Object)
write(Object)
serializeCacheKey


Threads
Focus: <No Selection>
Aspect: <No Selection>
Show concurrent:
Contained
Same threads
Time Range:
Set
1
Filter Events From 22:33:13:143 to 22:33:39:703
Reset
Thread State Selection
2
Log4j2-TF-1-AsyncLogger[Default
Log4j2-TF-2-AsyncLogger[AsyncC
HttpClient-1-SelectorManager
HttpClient-2-SelectorManager
idle-connection-reaper
ForkJoinPool.commonPool-worke
LaunchDarkly-event-delivery-45
LaunchDarkly-event-delivery-46
3
10/7/2025, 10:33:20.000 PM
10:33:25 PM
10:33:30 PM
10:33:35 PM
Stack Trace
Flame Graph
Samples
4
EntityLoader.load(UUID)
EntityLoader.load(Object)
CacheLoader.reload(Object, Object)
CacheLoader.lambda$asyncReload$2(Object, Object)


Garbage Collections
Focus: <No Selection>
Aspect: <No Selection>
Show concurrent:
Contained
Same threads
Time Range:
Set
Clear
GC ID
Cause
Collector Name
Longest Pause
Pause Phases
Metaspace
Heap
6 GiB
4 GiB
2 GiB
Heap Post GC
1 GiB
512 MiB
Metaspace
128 MiB
64 MiB
Longest Pause
20 ms
Sum of Pauses
20 ms
Pause Phases
10/7/2025
10:33:00 PM
10:33:15 PM
10:33:30 PM
10:33:45 PM
Used Heap
Heap Space : Committed Size
Heap Space : Reserved Size
Used Heap Post GC
Metaspace : Used
Metaspace : Committed
Metaspace : Reserved
Longest Pause
Sum of Pauses
Pause Phases
Thread Activity


Lock Instances
Focus: <No Selection>
Aspect: <No Selection>
Show concurrent:
Contained
Same threads
Monitor Class
Total Blocked Time
Distinct Threads
Count
java.util.concurrent.ConcurrentHashMap$ReservationNode
96.627 ms
1
1
jdk.jfr.internal.PlatformRecorder
13.678 ms
1
1
1
Monitor Address
Total Blocked Time
Distinct Threads
Count
0xFFFEFC218D20
96.627 ms
1
1
0xFFFF15DD8C10
13.678 ms
1
1
Thread
Total Blocked Time
Count
2
http-nio-8080-exec-58
96.627 ms
1
JFR Periodic Tasks
13.678 ms
1
Stack Trace
Flame Graph
Samples
Stack Trace
Samples
Percentage
3
Object java.util.concurrent.ConcurrentHashMap.compute(Object, BiFunction)
1
100 %
Object com.github.benmanes.caffeine.cache.BoundedLocalCache.doComputeIfAbsent(Object, Object, Function, lo...
1
100 %
Object com.github.benmanes.caffeine.cache.BoundedLocalCache.computeIfAbsent(Object, Function, boolean, bool...
1
100 %
Object com.github.benmanes.caffeine.cache.LocalCache.computeIfAbsent(Object, Function)
1
100 %


Method Profiling
Focus: <No Selection>
Aspect: <No Selection>
Show concurrent:
Contained
Same threads
Method
Count
Percentage
org.apache.tomcat.util.buf.MessageBytes.equalsIgnoreCase(String) 152 8.72 %
java.util.HashMap.getNode(Object) 73 4.19 %
java.io.ByteArrayOutputStream.ensureCapacity(int) 70 4.02 %
org.apache.lucene.store.DataInput.readVLong(boolean) 64 3.67 %
java.util.concurrent.ConcurrentLinkedQueue.size() 48 2.75 %
1
Predecessors
Successors
Stack Trace
Count
Percentage
2
int java.util.concurrent.ConcurrentLinkedQueue.size() 48
void pl.codewise.utils.id.aspects.impl.ClusteredPseudoRandomIdGenerator.nextId(BitsBuffer) 48
CompositeId pl.codewise.utils.id.impl.Base32V2CompositeIdServiceImpl.newId() 48
void pl.codewise.bclig.traffic.logic.packetsender.bundler.PacketBundler.setIdAndAdd(RequestData) 48
Stack Trace
Flame Graph
Samples
3
ConcurrentLinkedQueue.size()
ClusteredPseudoRandomIdGenerator.nextId(BitsBuffer)
Base32V2CompositeIdServiceImpl.newId()
PacketBundler.setIdAndAdd(RequestData)
VisitHandler.countVisit(VisitRequest, HttpServletResponse, CampaignData, VisitData)
size()
size()
size()
nextId
nextId
newId
setId...
setId...

async-profiler

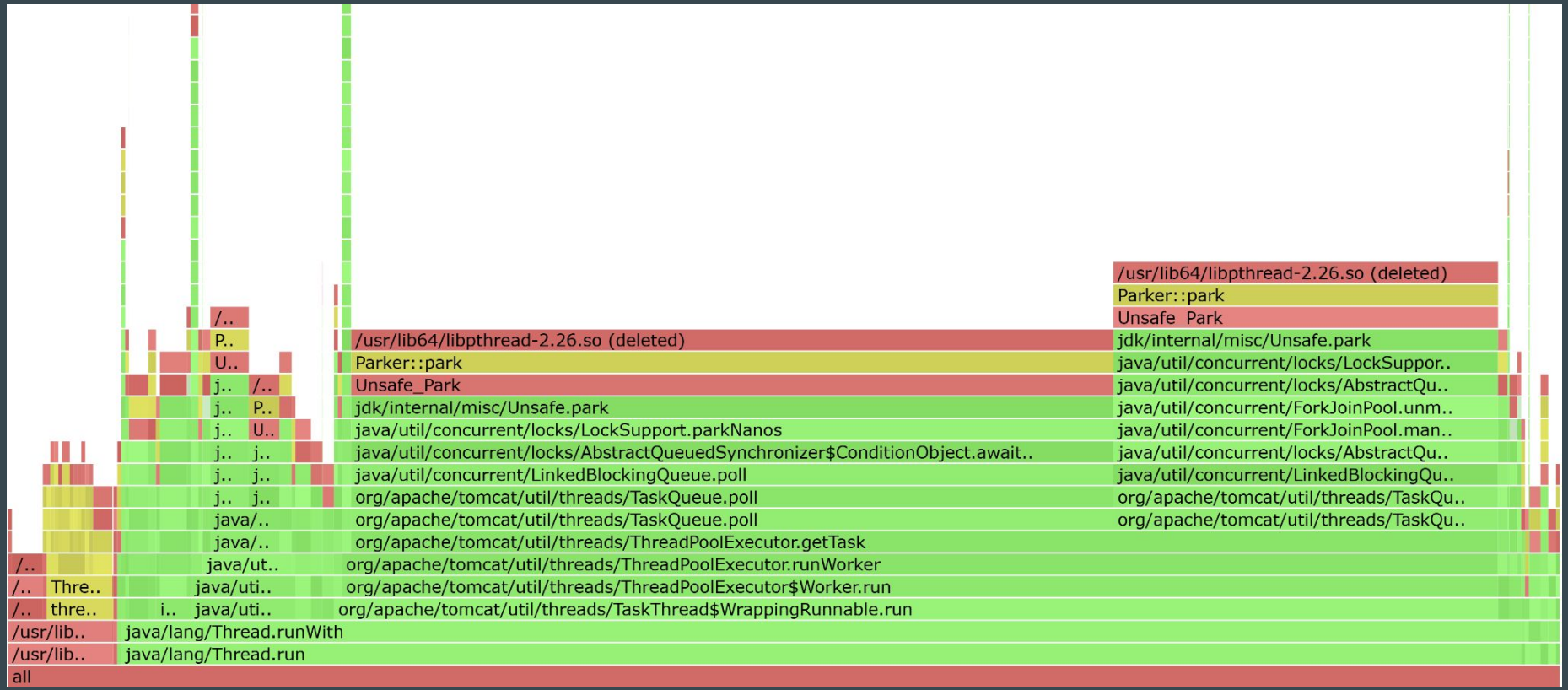
- A low-overhead sampling profiler for Java
- Immune to the “Safepoint bias problem”
  - Uses native stack walking to capture both Java and native stack traces, ensuring a more accurate representation of the application's execution
- Monitors non-Java threads (e.g., GC and JIT compiler threads) and shows native and kernel frames in stack traces
- Maintained by the Amazon Corretto team

3 hours of YouTube presentation on async-profiler under the hood and how to use

async-profiler

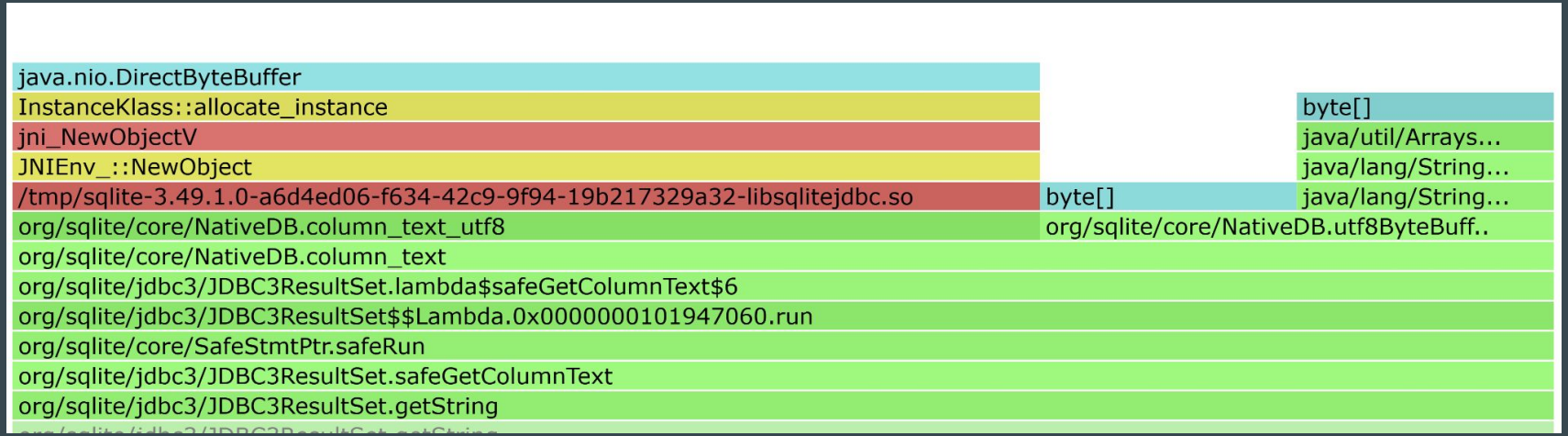
- As easy as
  - `$ wget https://github.com/async-profiler/async-profiler/releases/download/v4.1/async-profiler-4.1-linux-arm64.tar.gz`
  - `$ cd async-profiler-4.1-linux-arm64/bin/`
  - `$ ./asprof -d 30 -f out.html jps`
- Shipped with JDK
  - `$ $JAVA_HOME/bin/asprof -d 30 -f out.html jps`
- Output
  - Collection of call stacks in text
  - Flame graphs in HTML
  - JFR recordings
- Explore
  - `$ ./asprof –help`
- It’s available in IntelliJ

- `$ ./asprof -d 30 -f out.html -e wall jps`
  - It's useful when profiling app startup - an agent must be attached upon run



async-profiler

- `$ ./asprof -d 30 -f out.html -e alloc jps`
  - Records and shows only allocation outside TLABs

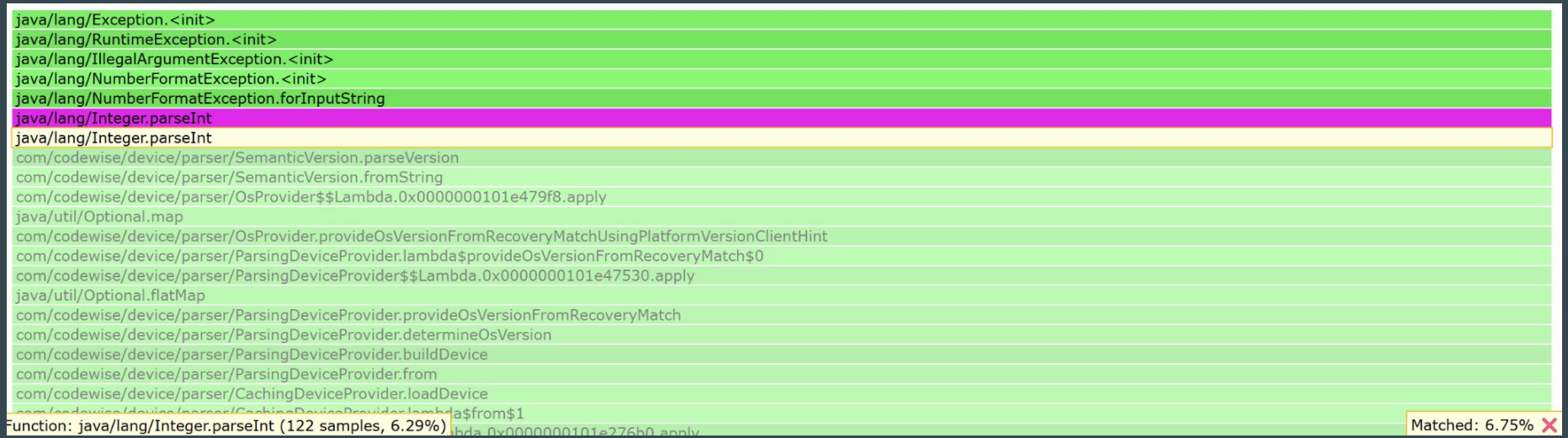


JVM Anatomy Quark #4: TLAB allocation

- $ ./asprof -d 30 -f out.html -e lock –total jps



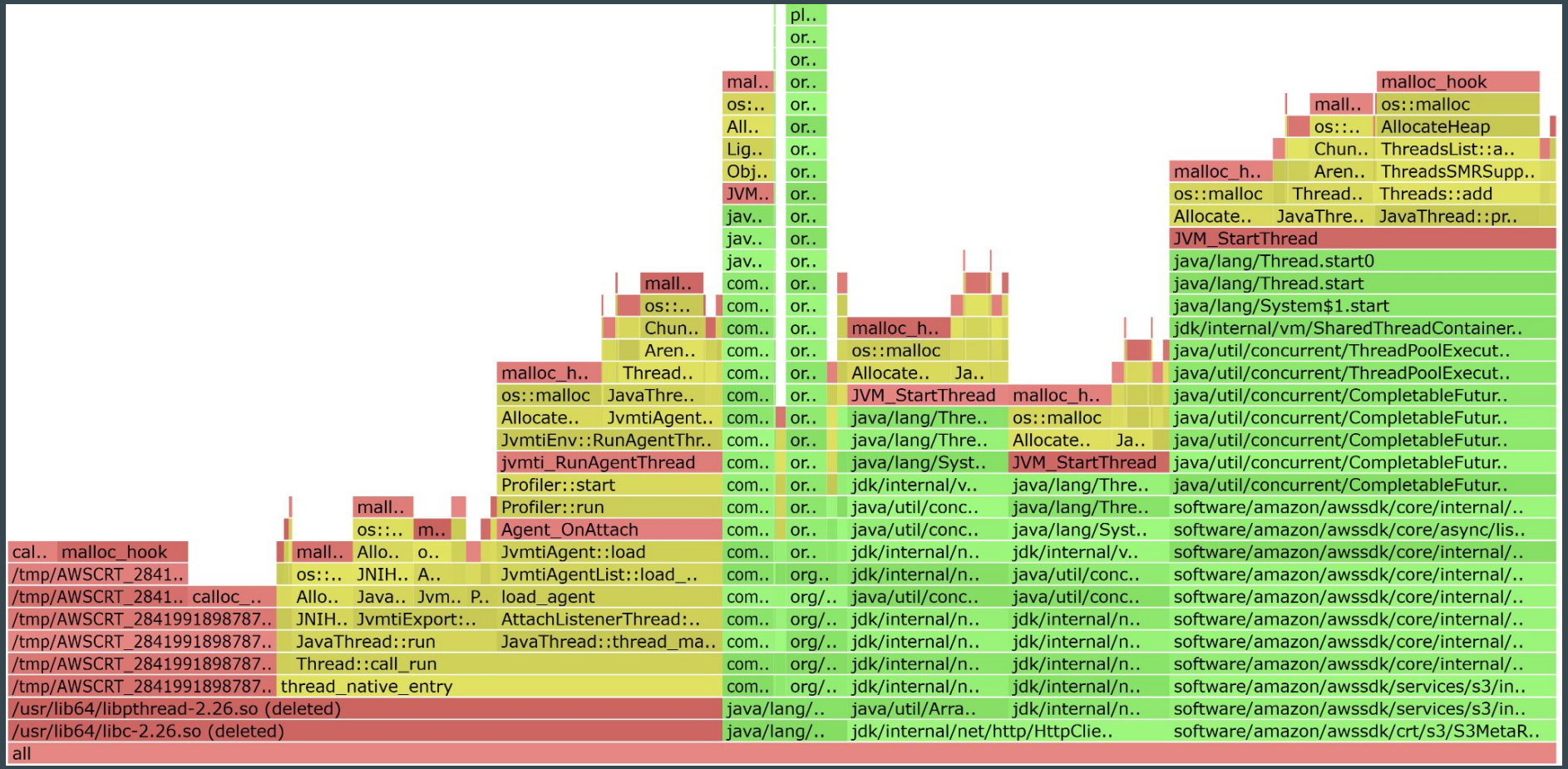
- $ ./asprof -d 30 -f out.html -e ‘java.lang.Exception.*’ jps



- $ ./asprof -d 30 -f out.jfr -e nativemem jps



- $ ./asprof -d 30 -f out.html -e nativemem --total jps
- $ ./jfrconv --total --nativemem --leak out.jfr out.html




Break It, Find It, Fix It: Hunting Memory Leaks and
Performance Bottlenecks

Thank you!